DOCKER

NGINX

23 DE OCTUBRE DE 2026

JOAQUIN VALENTIN OTTONELLO
DAW2

INDICE

INTRODUCCIÓN 4
DESARROLLO 5
INSTALACIÓN NGINX EN DOCKER 6
COMPROBACIÓN NGINX 8
PAGINA WEB EJEMPLO MEDIANTE JSP Y SERVLET.JAVA EN TOMCAT (.WAR) 10
CODIGO 16

UD2 - Práctica 3. Comenzando con contenedores

En esta práctica, vamos a comenzar a utilizar contenedores de la mano de Docker. Recordemos que Docker es una plataforma de virtualización ligera que permite crear, distribuir y ejecutar contenedores. Su objetivo principal es que las aplicaciones se ejecuten de manera consistente en cualquier entorno, ya sea tu ordenador, un servidor o la nube.

Para ello, vamos a descargar e instalar Docker Desktop desde este enlace. Durante la instalación, habilita WSL 2 si se te solicita. Hecho esto, podemos abrir PowerShell para comprobar que Docker funciona:

docker --version

Si nos devuelve la versión instalada, significa que nuestra instalación ha sido correcta. Ahora vamos a realizar una pequeña prueba utilizando el servidor Nginx. Para ello, accedemos a la carpeta Docker (situada en C:\Archivos de Programa) y crearemos una carpeta llamada "*minginx*".

Hecho esto, crearemos una página sencilla (ha de contener al menos un HTML y un CSS) y los guardamos en la carpeta que hemos creado. Para ejecutar el contenedor, vuelve a abrir PowerShell desde la carpeta que acabas de crear y ejecuta el siguiente comando:

docker run --name mi-nginx -v ${PWD}\index.html:/usr/share/nginx/html/index.html:ro -p 8080:80 -d nginx

Si todo ha funcionado correctamente, deberías poder ver tu web desde la dirección http://localhost:8080. A continuación, investiga como podemos realizar las siguientes tareas básicas con nuestro contenedor y demuestra su funcionamiento (bien sea a través de la interfaz gráfica o usando la línea de comandos de PowerShell):

- Para el contenedor
- Reinicia el contenedor
- Muestra los logs del contenedor
- Borra el contenedor

¿Qué hacer ahora?

1. Crea un informe con tu plantilla en el que expliques cómo has completado tu actividad. Debe incluir, al menos, los siguientes elementos:
    a. Portada
    b. Índice
    c. Capturas de pantalla y explicaciones sobre:
        i. El funcionamiento de tu contenedor, especialmente los ficheros utilizados
        ii. El despliegue de tu web en el contenedor iii. Las tareas solicitadas y su correcto funcionamiento
2. Responde brevemente a las siguientes preguntas:
    a. ¿Qué es Nginx? ¿Qué ventajas aporta sobre otras soluciones similares ya trabajadas en clase?
    b. "Disecciona" la instrucción proporcionada anteriormente para lanzar el contenedor. ¿Qué utilidad tiene cada parte de la instrucción?
    c. ¿Has tenido que instalar Nginx en tu equipo para usar el contenedor? ¿Puedes explicar este principio de funcionamiento de Docker?

SOLO SE ACEPTARÁN ARCHIVOS PDF. CUALQUIER .DOCX, .DOC U OTROS NO SERÁN EVALUADOS

INTRODUCCIÓN

En este documento se explayará una documentación, sobre como montar NGINX en un contenedor de Docker, además de resolver algunas preguntas, así como, hacer énfasis en las ventajas y desventajas de usar este servidor (nginx).

DESARROLLO

En esta práctica se tratará de levantar NGINX mediante comandos de manera que el resultado (servidor NGINX), resulte alojado en docker.

Primero empezaremos por instalar docker desktop.



Aunque no encontré otra imagen, ya que la instalación personalmente ya la llevé a cabo; es seleccionar la segunda opción y marcar “usar wsl2 en vez de HyperV”

Una vez terminada la instalación de docker, si la volvemos a abrir no0s dirá que falta wsl, con lo cual procederá a abrirnos una terminal donde instalar wsl (por experiencia personal, es casi mejor idea cerrar docker y todo, y abrir una terminal por tu cuenta y poner “wsl”, darle a intro y se comenzará a descargar, de esta manera no habrá errores, cierres inesperados, etc etc)

INSTALACIÓN NGINX EN DOCKER

Aunque en la práctica sale, entrar a “C:\ Programs\Docker” y crear una carpeta mi-nginx, y después ejcutar esto:

docker run --name mi-nginx -v ${PWD}\index.html:/usr/share/nginx/html/index.html:ro -p 8080:80 -d nginx

No funciona, no porque se me canta decirlo, o sienta la necesidad, sino por;



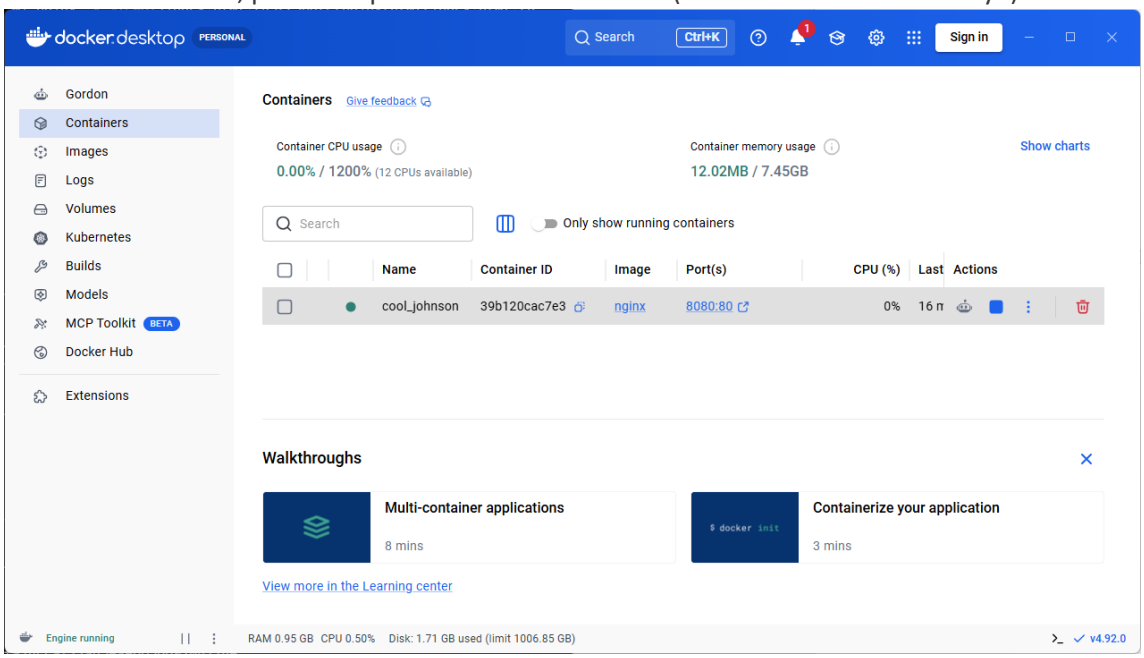
```
PS C:\Users\daw2> docker run --name mi-nginx -v ${PWD}\index.html:/usr/share/nginx/html/index.html:ro
-p 8080:80 -d nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
46243d3234ed: Pull complete
f1169c633cbc: Pull complete
3326c3817340: Pull complete
f802f27d954b: Pull complete
2056b40bae09: Pull complete
afa8dec48454: Pull complete
6b37362b3da7: Pull complete
37d8c7707e42: Download complete
e40088050cb6: Download complete
Digest: sha256:abe47724e466aeab9a345d8e46a221c2fa8953c7848bb4a3bd9976a7199f8cf2
Status: Downloaded newer image for nginx:latest
3cf710e4cf0fdbeb2dfa90cefefbaf3ca690ce9645290425d382f02b9988ebdc
docker: Error response from daemon: failed to create task for container: failed to create shim task: O
CI runtime create failed: runc create failed: unable to start container process: error during containe
r init: error mounting "/run/desktop/mnt/host/c/Users/daw2/index.html" to rootfs at "/usr/share/nginx/
html/index.html": mount src=/run/desktop/mnt/host/c/Users/daw2/index.html, dst=/usr/share/nginx/html/i
ndex.html, dstFd=/proc/thread-self/fd/14, flags=MS_BIND|MS_REC: not a directory: Are you trying to mou
nt a directory onto a file (or vice-versa)? Check if the specified host path exists and is the expecte
d type
```

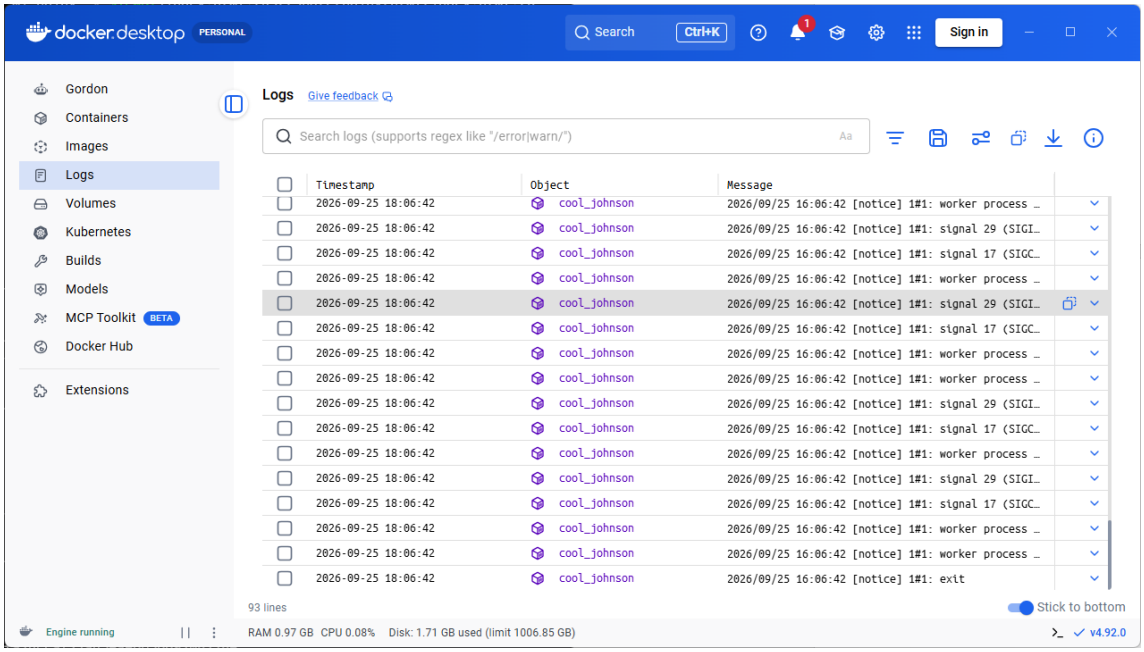
Aunque se instala la imagen da error de docker

Acabé haciendo: *docker run –p 8080:80 –d nginx*

```
PS C:\Users\daw2> docker run -p 8080:80 -d nginx
39b120cac7e3c2886896c1e0d5e554883b6fa24ccaf27a45eae6b5669208228e
```

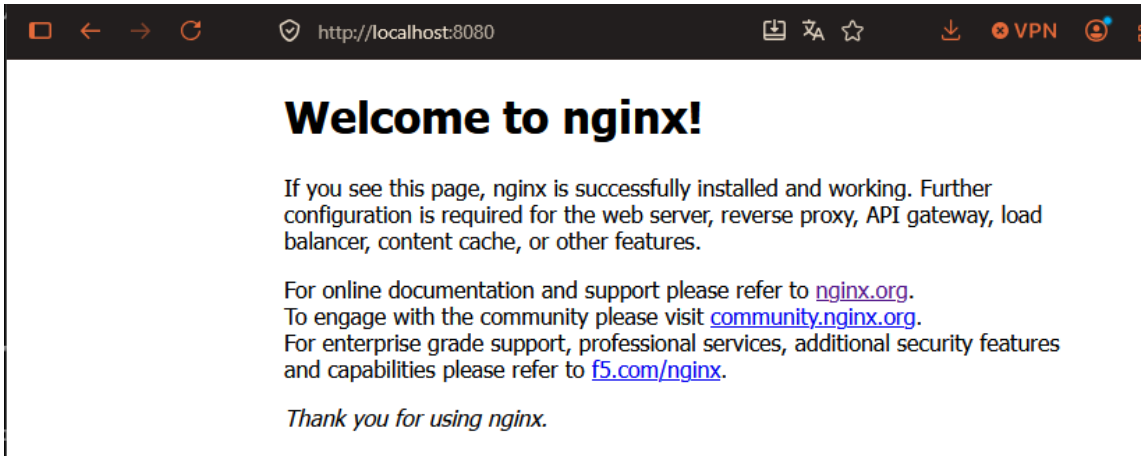
Una vez hecho esto, podemos proceder a iniciar docker (si no lo teníamos abierto ya)





COMPROBACIÓN NGINX

Y por último, se procederá a abrir en el navegador http://localhost:8080 (que es el puerto donde se despliega casi todo lo que sea local, o este en desarrollo)



a. ¿Qué es Nginx? ¿Qué ventajas aporta sobre otras soluciones similares ya trabajadas en clase?

Nginx (pronunciado *"engine-ex"*) es un servidor web, **proxy inverso** y **balanceador de carga** de código abierto, creado en 2004 por Igor Sysoev para resolver el problema C10k (gestionar 10.000+ conexiones simultáneas).

Frente a **Apache** (la solución más habitual trabajada en clase), sus ventajas principales son:

Aspecto	Nginx	Apache
Arquitectura	Asíncrona, basada en eventos	Basada en procesos/hilos (uno por petición)
Consumo de memoria	Muy bajo	Mayor (un hilo/proceso por conexión)
Contenido estático	Excelente rendimiento	Correcto, pero menos eficiente
Concurrency	Miles de conexiones simultáneas con pocos recursos	Escala peor bajo alta carga
Funciones extra	Proxy inverso, balanceo de carga, caché HTTP integrados	Requiere módulos adicionales

En resumen: Nginx es **más ligero, escalable y eficiente** para alto tráfico y contenido estático, mientras que Apache es más flexible para contenido dinámico (módulos `.htaccess`, PHP integrado, etc.).

b. "Disecciona" la instrucción proporcionada anteriormente para lanzar el contenedor. ¿Qué utilidad tiene cada parte de la instrucción?

docker run -d --name my-nginx -p 8080:80 -v $(pwd):/usr/share/nginx/html nginx

Parte	Utilidad
docker run	*Crea y arranca un nuevo contenedor a partir de una imagen.*
-d	***Detached mode**: ejecuta el contenedor en segundo plano y libera la terminal.*
--name my-nginx	*Asigna un nombre legible al contenedor para referenciarlo fácilmente en comandos posteriores (`docker stop`, `docker logs`, etc.).*
-p 8080:80	***Publica un puerto**: mapea el puerto 8080 del host al puerto 80 del contenedor. Así, al navegar a `http://localhost:8080`, el tráfico llega a Nginx dentro del contenedor.*
-v $(pwd):/usr/share/nginx/html	***Monta un volumen**: une un directorio local del host con el directorio donde Nginx sirve archivos (`/usr/share/nginx/html`). Permite editar los archivos HTML localmente y ver los cambios sin reiniciar el contenedor.*

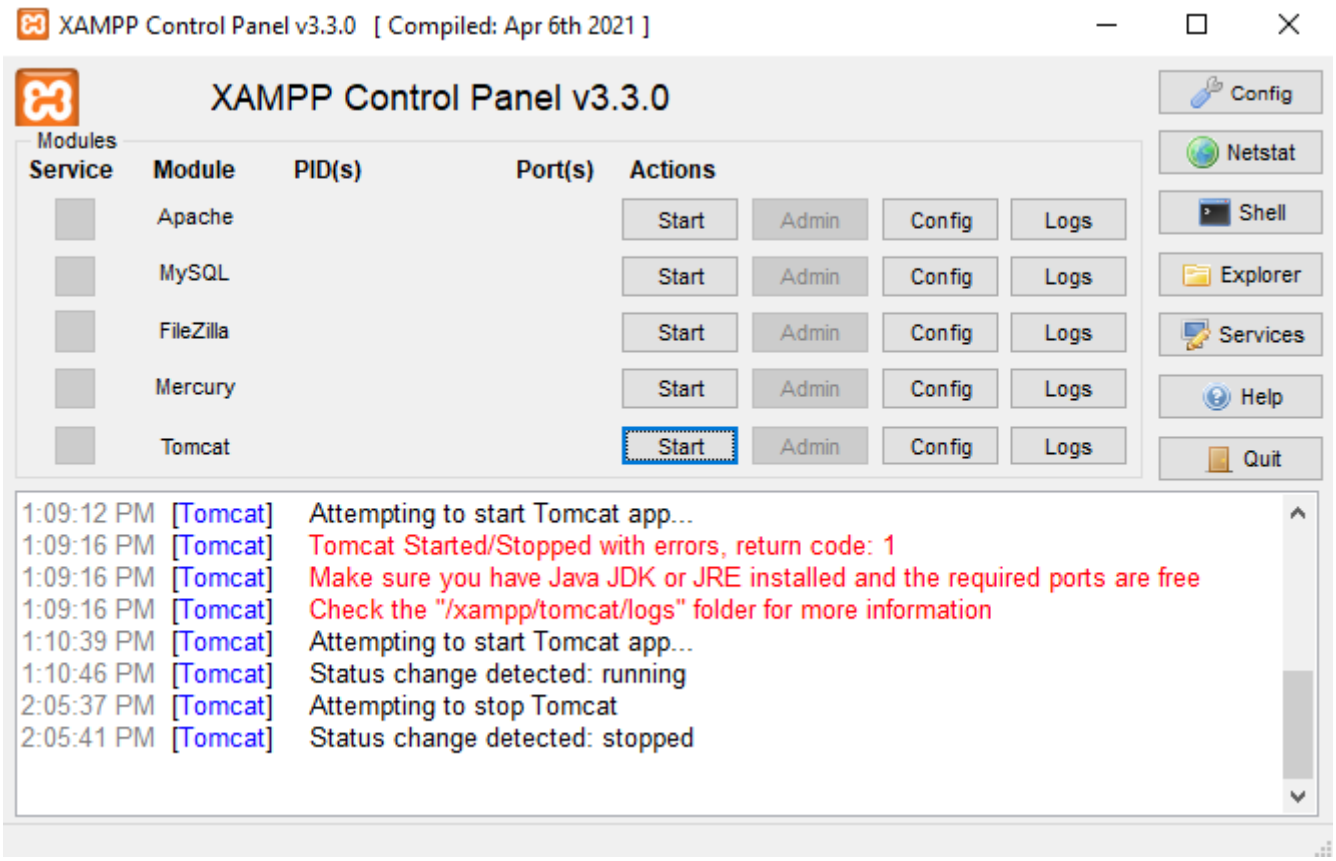
nginx	***Imagen a usar**: especifica la imagen de Nginx (la descargará de Docker Hub si no existe localmente).*

c. ¿Has tenido que instalar Nginx en tu equipo para usar el contenedor? ¿Puedes explicar este principio de funcionamiento de Docker?

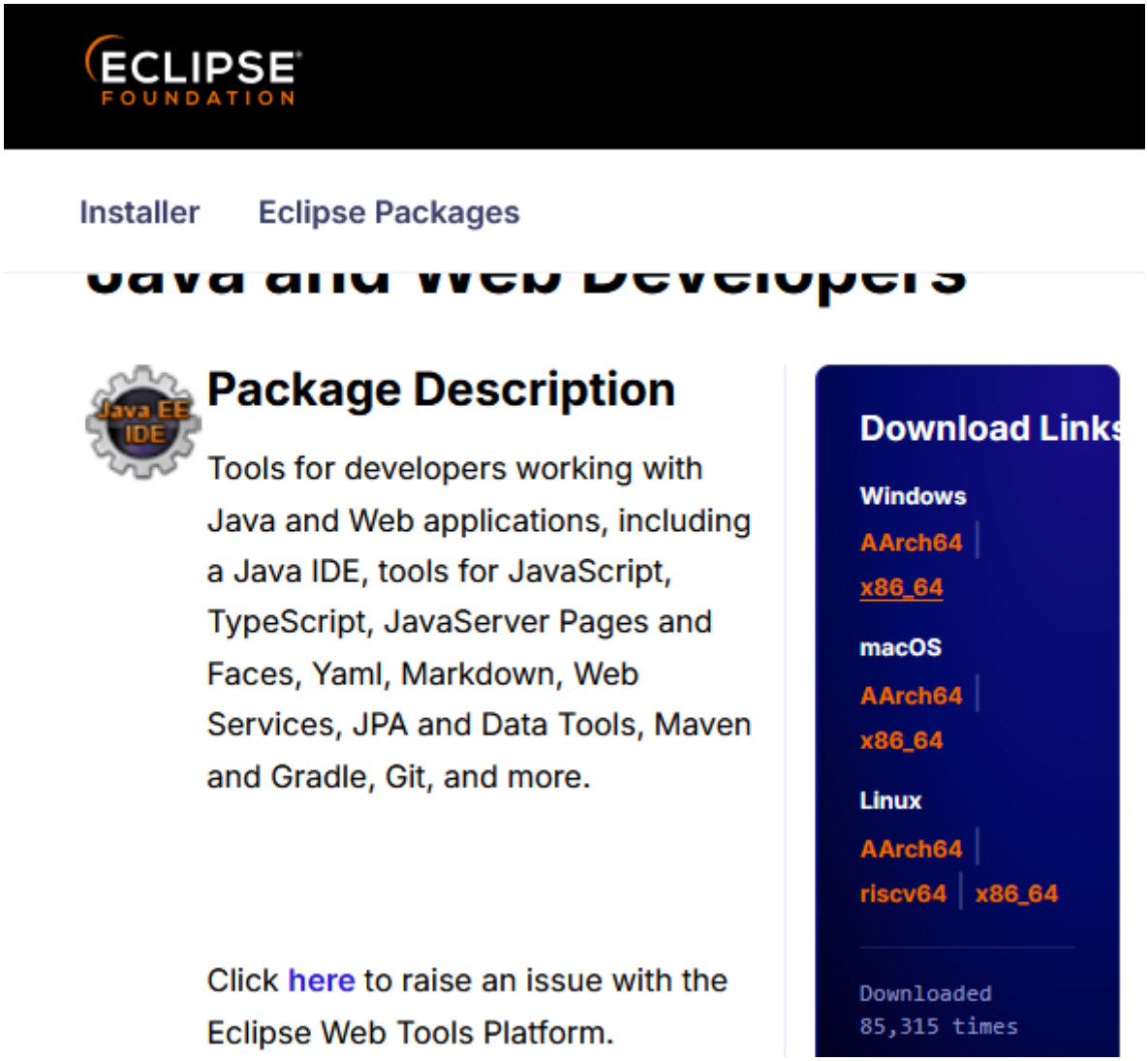
El contenedor es una unidad autocontenida que incluye la aplicación (Nginx) junto con **todas sus dependencias** (librerías, binarios, configuración) empaquetadas en la imagen.

PAGINA WEB EJEMPLO MEDIANTE JSP Y SERVLET.JAVA EN TOMCAT (.WAR)

Aunque en la práctica se pide usar el TomCCat de XAMP:



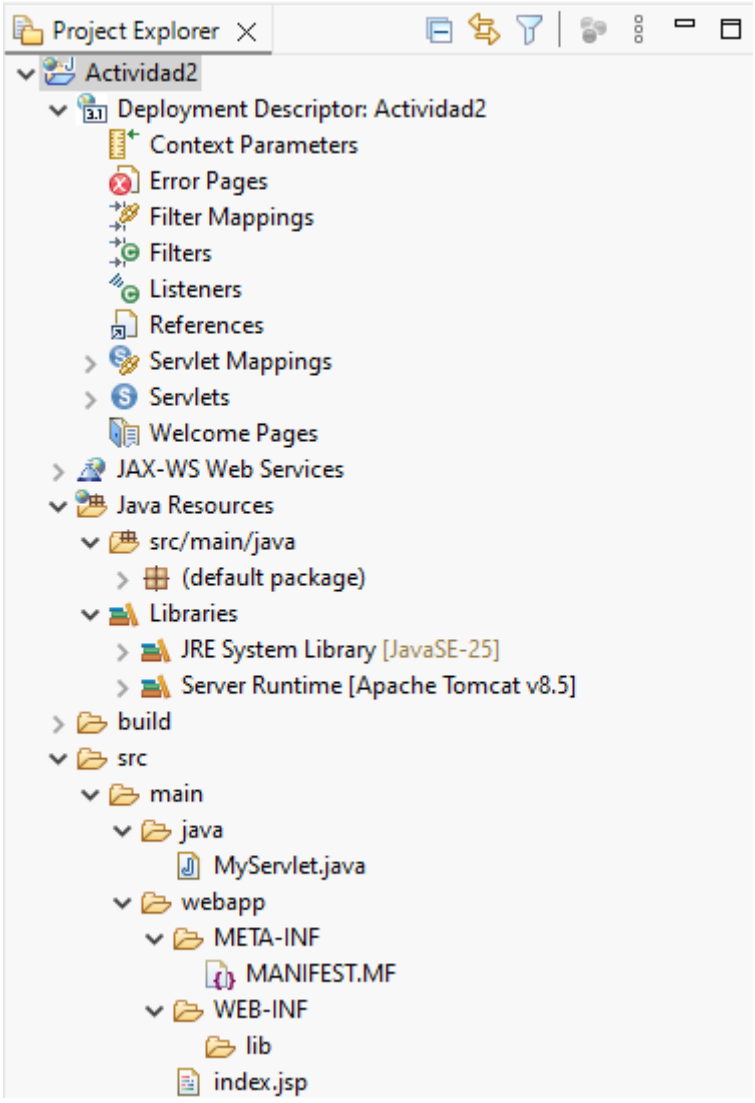
Levanté el propio servidor TomCat de Eclipse (versión Web Developer)[https://www.eclipse.org/downloads/packages/release/2026-09/r/eclipse-ide-enterprise-java-and-web-developers]



Y descargar el paquete de java

[https://www.java.com/es/download/manual.jsp]




Project Explorer
Actividad2
Deployment Descriptor: Actividad2
Context Parameters
Error Pages
Filter Mappings
Filters
Listeners
References
Servlet Mappings
Servlets
Welcome Pages
JAX-WS Web Services
Java Resources
src/main/java
(default package)
Libraries
JRE System Library [JavaSE-25]
Server Runtime [Apache Tomcat v8.5]
build
src
main
java
MyServlet.java
webapp
META-INF
MANIFEST.MF
WEB-INF
lib
index.jsp

.project | MyServlet.java | index.jsp

```jsp
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>

<!DOCTYPE html>
<html>

<head>
    <meta charset="ISO-8859-1">
    <title>Activity 3</title>
</head>

<body>

    <p>Hello from the JSP file!</p>

    <form action="Actividad2" method="GET">
    <button type="submit">Get message</button>
</form>

<br>

<form action="Actividad2" method="POST">
    <button type="submit">Post message</button>
</form>

</body>

</html>
```

.project | MyServlet.java | index.jsp

```java
import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/Actividad2")
public class MyServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    public MyServlet() {
        super();
    }

    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

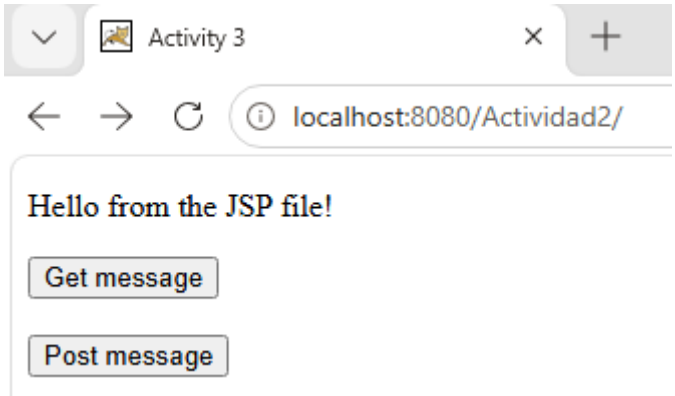
        out.println("<h1>Message from doGet method from MyServlet</h1>");
    }

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        out.println("<h1>Message from doPost method from MyServlet</h1>");
    }
}
```







Se cambia el código a un ejemplo de obtención de grados Celsius, para su posterior conversión a grados kelvin (Celsius + 273)





CODIGO

.project | MyServlet.java | index.jsp

```java
import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/Actividad2")
public class MyServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    public MyServlet() {
        super();
    }

    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html;charset=UTF-8");

        PrintWriter out = response.getWriter();

        String temperatura = request.getParameter("temperatura"); //obtiene parámetro del jsp

        if (temperatura != null && !temperatura.isEmpty()) {

            double temp = Double.parseDouble(temperatura); //lo transforma a un número decimal

            String resultado = calc(temp); //se llama a una función externa para calcular los kelvin


            //-------ESTRUCTURA DE COMO SE MUESTRA EL RESULTADO------

                //SI LA TEMPERATURA NO ES NULA O NO SE INSERTA TEMPERATURA
            out.println("<html>");
            out.println("<body>");
            out.println("<h1>Resultado</h1>");
            out.println("<p>" + resultado + "</p>");
            out.println("</body>");
            out.println("</html>");

        } else { //O SI ES NULA SE PREGUNTA
```

```java
            out.println("<html>");
            out.println("<body>");
            out.println("<h1>Input a temperature (Cº):</h1>");
            out.println("</body>");
            out.println("</html>");
        }
    }

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html;charset=UTF-8");

        PrintWriter out = response.getWriter();

        out.println("<html>");
        out.println("<body>");
        out.println("<h1>Message from doPost method from MyServlet</h1>");
        out.println("</body>");
        out.println("</html>");
    }

    protected String calc(double temp) {

        double kelvin = temp + 273;

        return "Conversión de grados Celsius " + temp
                + ", a Kelvin: " + kelvin;
    }
}
```

.project | MyServlet.java | index.jsp

```jsp
<%@ page language="java"
    contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>

<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Actividad 2</title>
</head>

<body>

    <h1>Conversión de temperatura</h1>

    <form action="Actividad2" method="get">

        <label>Temperatura en Celsius:</label>

        <input type="number"
               name="temperatura"
               step="any"
               required>

        <input type="submit" value="Convertir">

    </form>

</body>
</html>
```